

实验三 Spring Boot 过滤器、拦截器与监听器

一、实验目的

1. 掌握 Spring Boot 中过滤器（Filter）、拦截器（Interceptor）和监听器（Listener）的基本概念与作用。
2. 理解三者请求处理生命周期中的执行顺序和适用场景。
3. 能够独立实现日志记录、权限校验、请求参数预处理等常见功能。
4. 学会通过监听器监听 Web 应用中的关键事件（如上下文初始化、会话创建等）。

二、实验要求

1. 熟悉 Spring Boot 基本开发环境（Maven/Gradle + IntelliJ IDEA/Eclipse）。
2. 掌握 Servlet API 和 Spring MVC 的核心概念。
3. 实验代码需包含完整的注释，最终提交可运行的项目代码和实验报告。
4. 实验过程中需记录关键步骤的截图或日志输出。

三、实验内容与步骤

1. 基础实验

任务 1：实现一个简单的日志过滤器

创建 Spring Boot Web 项目，添加 spring-boot-starter-web 依赖。

编写自定义过滤器 LoggingFilter，在控制台输出请求的 URL、方法和耗时。

```
@Component
public class LoggingFilter implements Filter {
    @Override
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
        throws IOException, ServletException {
        long startTime = System.currentTimeMillis();
        chain.doFilter(request, response);
        long endTime = System.currentTimeMillis();
        System.out.println("Request URL: " + ((HttpServletRequest) request).getRequestURL());
        System.out.println("Time taken: " + (endTime - startTime) + "ms");
    }
}
```

访问 /hello 接口，观察控制台输出。

任务 2：实现请求头校验拦截器

创建自定义拦截器 `HeaderInterceptor`，校验请求头中是否包含 `X-Auth-Token`。

```
@Component
public class HeaderInterceptor implements HandlerInterceptor {
    @Override
    public boolean preHandle(HttpServletRequest request, HttpServletResponse
response, Object handler)
        throws Exception {
        String token = request.getHeader("X-Auth-Token");
        if (token == null || token.isEmpty()) {
            response.setStatus(401);
            return false; // 中断请求
        }
        return true;
    }
}
```

在配置类中注册拦截器：

```
@Configuration
public class WebMvcConfig implements WebMvcConfigurer {
    @Autowired
    private HeaderInterceptor headerInterceptor;

    @Override
    public void addInterceptors(InterceptorRegistry registry) {
registry.addInterceptor(headerInterceptor).addPathPatterns("/**");
    }
}
```

测试带/不带 `X-Auth-Token` 的请求，观察拦截效果。

2.进阶实验

任务 3：结合监听器实现在线用户统计

创建监听器 `OnlineUserListener`，监听 `HttpSessionEvent` 统计在线人数：

```
@WebListener
public class OnlineUserListener implements HttpSessionListener {
```

```

private static int onlineCount = 0;

@Override
public void sessionCreated(HttpSessionEvent se) {
    onlineCount++;
}

@Override
public void sessionDestroyed(HttpSessionEvent se) {
    onlineCount--;
}

public static int getOnlineCount() {
    return onlineCount;
}
}

```

创建控制器 /userCount 返回当前在线人数：

```

@RestController
public class UserController {
    @GetMapping("/userCount")
    public String getUserCount() {
        return "Online users: " + OnlineUserListener.getOnlineCount();
    }
}

```

通过多开浏览器窗口测试在线人数变化。

3.实战实验

任务 4：综合应用实现 API 防护

分别结合过滤器、拦截器和监听器实现以下功能：

过滤器：统一处理跨域请求（CORS）

拦截器：校验用户角色（如 ADMIN 角色才能访问 /admin 接口）

监听器：记录接口调用次数到内存或文件

示例代码片段（角色校验拦截器）：

```

@Override
public boolean preHandle(HttpServletRequest request, HttpServletResponse
response, Object handler) {
    if (!request.getRequestURI().startsWith("/admin")) return true;
    String role = (String) request.getSession().getAttribute("role");
}

```

```
        if (!"ADMIN".equals(role)) {  
            response.setStatus(403);  
            return false;  
        }  
        return true;  
    }  
}
```

四、实验提问

1. 过滤器、拦截器和监听器在请求处理流程中的执行顺序是什么？请画示意图说明。
2. 如果需要在请求到达 Controller 方法前修改请求参数，应该选择哪种技术实现？监听器能否直接修改响应结果？为什么？
3. 如何避免拦截器对静态资源（如 CSS/JS 文件）的干扰？